

Euclidean Algorithm Exercises

Q&A Notes

Question 1

Suppose

$$a = p_1^{e_1} p_2^{e_2} \cdots p_r^{e_r} \quad \text{and} \quad b = p_1^{f_1} p_2^{f_2} \cdots p_r^{f_r},$$

where missing primes are given exponent 0. Prove that

$$\gcd(a, b) = p_1^{\min(e_1, f_1)} p_2^{\min(e_2, f_2)} \cdots p_r^{\min(e_r, f_r)}.$$

Answer 1

The gcd takes exactly the smaller exponent of each prime.

Let d be any common divisor of a and b . If

$$d = p_1^{g_1} p_2^{g_2} \cdots p_r^{g_r},$$

then $d \mid a$ gives $g_i \leq e_i$, and $d \mid b$ gives $g_i \leq f_i$. Hence

$$g_i \leq \min(e_i, f_i)$$

for every i . Therefore every common divisor divides

$$p_1^{\min(e_1, f_1)} p_2^{\min(e_2, f_2)} \cdots p_r^{\min(e_r, f_r)}.$$

On the other hand, this number divides both a and b , because each exponent is no larger than the corresponding exponent in a and in b . So it is the greatest common divisor:

$$\gcd(a, b) = \prod_{i=1}^r p_i^{\min(e_i, f_i)}.$$

Question 2

Compute the values (d, x, y) returned by

ExtendedEuclid(899, 493).

Answer 2

The result is

$$(d, x, y) = (29, -6, 11).$$

So

$$29 = 899(-6) + 493(11).$$

Use the Euclidean algorithm:

$$899 = 493(1) + 406,$$

$$493 = 406(1) + 87,$$

$$406 = 87(4) + 58,$$

$$87 = 58(1) + 29,$$

$$58 = 29(2) + 0.$$

Thus

$$\gcd(899, 493) = 29.$$

Now back substitute:

$$29 = 87 - 58,$$

$$58 = 406 - 4(87),$$

so

$$29 = 87 - (406 - 4(87)) = 5(87) - 406.$$

Also

$$87 = 493 - 406,$$

so

$$29 = 5(493 - 406) - 406 = 5(493) - 6(406).$$

Finally,

$$406 = 899 - 493,$$

hence

$$29 = 5(493) - 6(899 - 493) = -6(899) + 11(493).$$

Therefore `ExtendedEuclid(899, 493)` returns

$$(29, -6, 11).$$

Question 3

Prove that for all integers a, k, n ,

$$\gcd(a, n) = \gcd(a + kn, n).$$

Answer 3

The two gcds are equal because a and $a + kn$ have exactly the same common divisors with n . Let $d \mid a$ and $d \mid n$. Since $d \mid n$, we have $d \mid kn$. Therefore

$$d \mid a + kn.$$

So every common divisor of a and n is also a common divisor of $a + kn$ and n .

Conversely, let $d \mid a + kn$ and $d \mid n$. Again $d \mid kn$, so

$$d \mid (a + kn) - kn = a.$$

Thus every common divisor of $a + kn$ and n is also a common divisor of a and n .

The two pairs have the same common divisors, so their greatest common divisors are equal:

$$\gcd(a, n) = \gcd(a + kn, n).$$

Question 4

Rewrite Euclid's algorithm in an iterative form that uses only a constant amount of memory.

Answer 4

Keep only the current pair (a, b) and one temporary remainder.

Iterative Euclid

```
EuclidIterative(a, b):  
    while b != 0:  
        r = a mod b  
        a = b  
        b = r  
    return a
```

At each loop iteration, the pair

$$(a, b)$$

is replaced by

$$(b, a \bmod b).$$

The gcd does not change because

$$\gcd(a, b) = \gcd(b, a \bmod b).$$

Only a , b , and r are stored, so the memory usage is constant:

$$O(1) \text{ integer variables.}$$

Question 5

If $a > b \geq 0$, show that $\text{Euclid}(a, b)$ makes at most

$$1 + \log_{\phi} b$$

recursive calls. Improve this bound to

$$1 + \log_{\phi} \left(\frac{b}{\gcd(a, b)} \right).$$

Answer 5

Let $\phi = (1 + \sqrt{5})/2$. The key fact is Lamé's Fibonacci bound:
If $a > b \geq 1$ and $\text{Euclid}(a, b)$ makes $k \geq 1$ recursive calls, then

$$b \geq F_{k+1}.$$

Also, Fibonacci numbers grow exponentially:

$$F_{k+1} \geq \phi^{k-1}.$$

Therefore,

$$b \geq F_{k+1} \geq \phi^{k-1}.$$

Taking $\log_{\phi}$ gives

$$\log_{\phi} b \geq k - 1,$$

so

$$k \leq 1 + \log_{\phi} b.$$

For the improved bound, let

$$g = \gcd(a, b).$$

Dividing both inputs by g does not change the sequence of quotients in Euclid's algorithm:

$$\text{Euclid}(a, b) \quad \text{and} \quad \text{Euclid}\left(\frac{a}{g}, \frac{b}{g}\right)$$

make the same number of recursive calls. Applying the previous bound to the reduced pair gives

$$k \leq 1 + \log_\phi\left(\frac{b}{g}\right) = 1 + \log_\phi\left(\frac{b}{\gcd(a, b)}\right).$$

Question 6

What does

$$\text{ExtendedEuclid}(F_{k+1}, F_k)$$

return? Prove your answer correct.

Answer 6

For $k \geq 2$, the return value is

$$\left(1, (-1)^{k+1}F_{k-2}, (-1)^kF_{k-1}\right).$$

That means

$$F_{k+1}((-1)^{k+1}F_{k-2}) + F_k((-1)^kF_{k-1}) = 1.$$

The gcd part is 1, since consecutive Fibonacci numbers are relatively prime:

$$\gcd(F_{k+1}, F_k) = 1.$$

For the coefficients, use this form of Cassini's identity:

$$F_k F_{k-1} - F_{k+1} F_{k-2} = (-1)^k.$$

Multiply both sides by $(-1)^k$:

$$(-1)^{k+1}F_{k+1}F_{k-2} + (-1)^kF_kF_{k-1} = 1,$$

which gives the coefficients

$$x = (-1)^{k+1}F_{k-2}, \quad y = (-1)^kF_{k-1}.$$

For example, when $k = 5$,

$$\text{ExtendedEuclid}(F_6, F_5) = \text{ExtendedEuclid}(8, 5)$$

returns

$$(1, 2, -3),$$

because

$$8(2) + 5(-3) = 1.$$

Question 7

Define the gcd function for more than two arguments by

$$\gcd(a_0, a_1, \dots, a_n) = \gcd(a_0, \gcd(a_1, \dots, a_n)).$$

Show that the result is independent of the order of the arguments. Also show how to find integers $x_0, x_1, \dots, x_n$ such that

$$\gcd(a_0, a_1, \dots, a_n) = a_0x_0 + a_1x_1 + \dots + a_nx_n.$$

Answer 7

The result is independent of order because the gcd is the largest integer that divides every input.

Let

$$g = \gcd(a_0, a_1, \dots, a_n).$$

Then g divides every a_i , and any integer that divides every a_i must divide g . This description does not depend on the order in which the inputs are written. Therefore the multi-argument gcd is order independent.

To find coefficients, combine the numbers one at a time using Extended Euclid.

Start with

$$D_0 = a_0.$$

Assume after processing $a_0, \dots, a_{i-1}$, we have

$$D_{i-1} = a_0x_0 + a_1x_1 + \dots + a_{i-1}x_{i-1}.$$

Run Extended Euclid on D_{i-1} and a_i :

$$D_i = \gcd(D_{i-1}, a_i) = uD_{i-1} + va_i.$$

Substitute the old expression for D_{i-1} :

$$D_i = u(a_0x_0 + \dots + a_{i-1}x_{i-1}) + va_i.$$

So the old coefficients are multiplied by u , and the new coefficient of a_i is v .

After all inputs are processed,

$$\boxed{\gcd(a_0, \dots, a_n) = a_0x_0 + \dots + a_nx_n.}$$

For the division count, each new argument costs a constant amount unless the current gcd strictly decreases. Whenever the current positive gcd strictly decreases, it becomes a proper divisor and is at most half as large. Hence there can be only $O(\lg M)$ such decreases, where

$$M = \max\{a_0, a_1, \dots, a_n\}.$$

Thus the total number of divisions is

$$\boxed{O(n + \lg M).}$$

Question 8

Show how to compute

$$\text{lcm}(a_1, a_2, \dots, a_n)$$

efficiently using the two-argument gcd operation as a subroutine.

Answer 8

For two positive integers,

$$\text{lcm}(a, b) = \frac{ab}{\text{gcd}(a, b)}.$$

It is usually better to compute it as

$$\text{lcm}(a, b) = \left(\frac{a}{\text{gcd}(a, b)} \right) b$$

to keep the intermediate value smaller.

For many inputs, fold from left to right:

LCM using GCD

```
LCM(a1, a2, ..., an):  
  L = a1  
  for i = 2 to n:  
    g = gcd(L, ai)  
    L = (L / g) * ai  
  return L
```

The invariant is:

$$L = \text{lcm}(a_1, \dots, a_i)$$

after processing a_i . Therefore, after the loop finishes,

$$L = \text{lcm}(a_1, \dots, a_n).$$

Question 9

Prove that n_1, n_2, n_3, n_4 are pairwise relatively prime if and only if

$$\text{gcd}(n_1 n_2, n_3 n_4) = 1$$

and

$$\text{gcd}(n_1 n_3, n_2 n_4) = 1.$$

Then explain the general idea for $n_1, n_2, \dots, n_k$.

Answer 9

First suppose n_1, n_2, n_3, n_4 are pairwise relatively prime. Then no prime divides two different n_i 's. Therefore no prime can divide both products $n_1 n_2$ and $n_3 n_4$, and no prime can divide both products $n_1 n_3$ and $n_2 n_4$. Hence both gcds are 1.

Conversely, suppose

$$\text{gcd}(n_1 n_2, n_3 n_4) = 1 \quad \text{and} \quad \text{gcd}(n_1 n_3, n_2 n_4) = 1.$$

The first gcd rules out common prime factors between any number in

$$\{n_1, n_2\}$$

and any number in

$$\{n_3, n_4\}.$$

So it rules out common factors for

$$(n_1, n_3), (n_1, n_4), (n_2, n_3), (n_2, n_4).$$

The second gcd rules out common factors for

$$(n_1, n_2), (n_1, n_4), (n_3, n_2), (n_3, n_4).$$

Together, every pair is covered. Thus every pair has gcd 1, so the four integers are pairwise relatively prime.

For k integers, assign each index a distinct binary label using

$$\lceil \lg k \rceil$$

bits. For each bit position, form two products: one product of all n_i 's whose label has bit 0, and one product of all n_i 's whose label has bit 1. Test whether the two products are relatively prime.

Any two distinct labels differ in some bit position, so every pair of numbers is separated by at least one such test. Therefore,

$\lceil \lg k \rceil$ gcd tests are enough to check pairwise relative primality.
--